

Computer Vision CS 236873 - Assignment 3

Daniel Moroz -
Murad Ektilat -

June 2025

1 Part 1: Sparse Reconstruction

In this part, we implemented a pipeline to perform sparse 3D reconstruction from two 2D images of a temple. This involved estimating the fundamental matrix, finding point correspondences across images, and triangulating 2D points to find their 3D coordinates.

1.1 Eight-Point Algorithm

1.1.1 Theory and Implementation

The goal of this function, `eight_point`, is to compute the Fundamental Matrix (F) from a set of at least 8 point correspondences. The fundamental matrix encapsulates the epipolar geometry between two views.

The implementation follows these key steps:

1. **Point Normalization:** To improve numerical stability, the input points (`pts1`, `pts2`) are normalized. Each coordinate is scaled by dividing by `pmax` (the maximum of the image's width and height). This is achieved using a transformation matrix T .
2. **Constructing the Constraint Matrix:** The epipolar constraint for a pair of corresponding points (p_1, p_2) is given by $p_2^T F p_1 = 0$. This equation is linear in the elements of F . By rearranging this for each of the N point pairs, we construct a homogeneous linear system $Af = 0$, where A is a $N \times 9$ matrix and f is the flattened 9×1 vector of the elements of F .
3. **Solving for F :** The system $Af = 0$ is solved using Singular Value Decomposition (SVD). The solution for f is the singular vector corresponding to the smallest singular value of A , which is the last column of the matrix V in the decomposition $A = U\Sigma V^T$.
4. **Enforcing the Rank-2 Constraint:** A valid fundamental matrix must have rank 2. The solution obtained via SVD might not satisfy this due to noise. We enforce this constraint by performing SVD on the computed F_{norm} , setting the smallest singular value to zero ($\sigma_3 = 0$), and recomputing $F'_{norm} = U\Sigma'V^T$.
5. **Refining and Un-normalizing:** The solution is optionally refined using local minimization (`refineF`) to minimize a geometric cost function. Finally, the matrix is un-normalized using the transformation $F_{unnorm} = T^T F'_{norm} T$ to bring it back to the original pixel coordinates.

1.1.2 Results

The computed Fundamental Matrix was used to visualize the epipolar lines for several selected points in the first image, as shown in Figure 1. The lines correctly pass through or very near the corresponding feature in the second image, validating our implementation.

1.2 Epipolar Correspondences

1.2.1 Theory and Implementation

The function `epipolar_correspondences` uses the fundamental matrix to find the precise location of a point from the first image (I_1) in the second image (I_2).

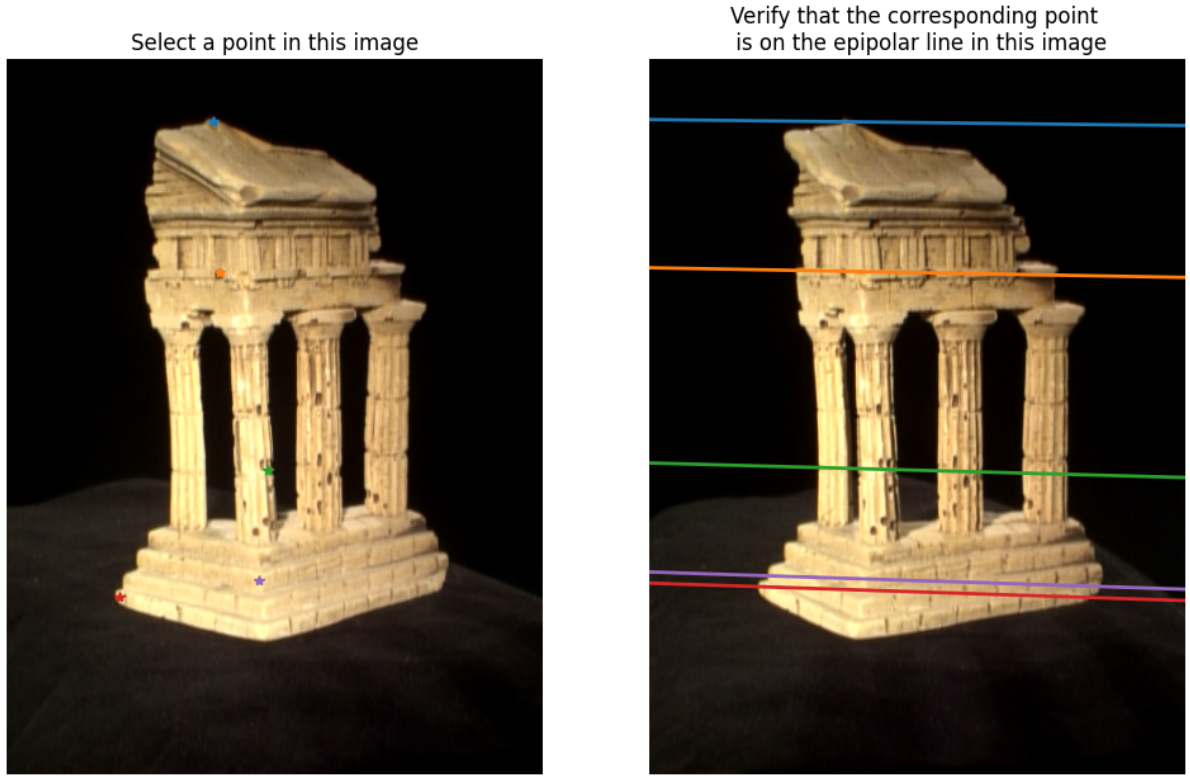


Figure 1: Visualization of epipolar lines using the computed Fundamental Matrix. Selected points in the left image correspond to the epipolar lines in the right image.

Similarity Metric: The matching process is based on patch similarity. For a point p in I_1 , we define a small square window (patch) of size 15×15 pixels around it. We then search for a matching patch in I_2 . To compare two patches, we use the **Sum of Squared Differences (SSD)** on grayscale intensity values. A lower SSD score indicates a better match.

The implementation steps are:

1. For a given point p_1 in I_1 , calculate the corresponding epipolar line l_2 in I_2 using the formula $l_2 = Fp_1$.
2. Sample a set of candidate points along the line l_2 in I_2 .
3. For each candidate point p_2 , extract a 15×15 patch around it.
4. Compute the SSD score between the patch from I_1 and the patch from each candidate in I_2 .
5. The corresponding point is the candidate point that yields the minimum SSD score.

1.2.2 Results

Figure 2 shows the results of our matching algorithm. The red dots on the right image, which represent the computed correspondences, lie correctly on their respective epipolar lines and accurately pinpoint the corresponding features.

Failure Cases: The algorithm failed (not accurate enough) in regions that are texture-less (blank wall area / harsh lighting) or contained high repetitive patterns (right most pillar). In these cases, the SSD metric is ambiguous, as many patches along the epipolar line will have similarly low scores.

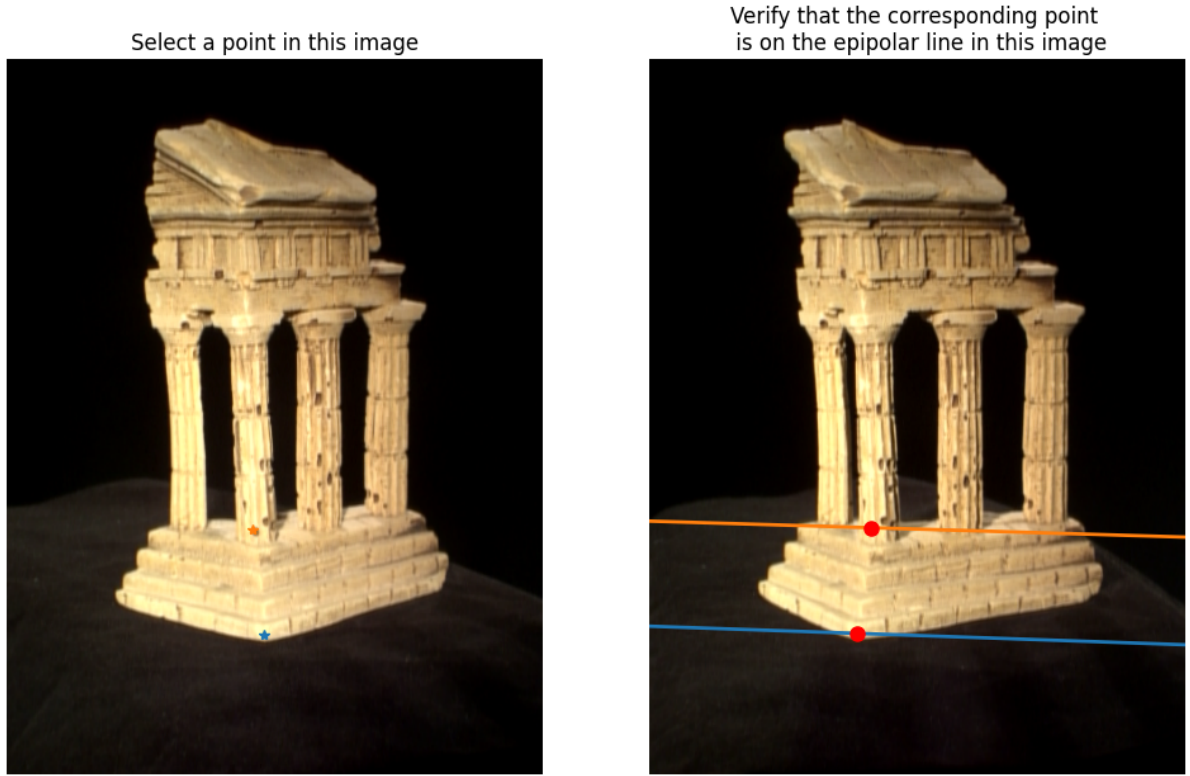


Figure 2: Visualization of computed epipolar correspondences. Red dots on the right image are the matches found by the algorithm for the points selected in the left image.

1.3 Essential Matrix

1.3.1 Theory and Implementation

The Essential Matrix (E) relates corresponding points in normalized camera coordinates, whereas the Fundamental Matrix (F) relates them in pixel coordinates. The relationship between them is defined by the camera intrinsic matrices, K_1 and K_2 . The function `essential_matrix` computes E using the formula:

$$E = K_2^T F K_1$$

1.3.2 Results

Using the previously computed F and the provided intrinsics, the estimated Essential Matrix for the temple image pair is:

```
Essential Matrix E:
[[ -1.34574426  29.07118462   7.75265911]
 [  68.88731584  -0.54273929 -370.14770947]
 [   1.17198847  374.71223603   0.4291653 ]]
```

1.4 Triangulation

1.4.1 Theory and Implementation

The `triangulate` function reconstructs the 3D coordinates of a point given its 2D projections in two images and the corresponding camera projection matrices (M_1, M_2). For each 2D-3D correspondence, we can set up a linear system of equations $Ax = 0$ and solve for the 3D point's homogeneous coordinates using SVD, similar to the Eight-Point Algorithm.

Determining the Correct Extrinsic Matrix: The Essential Matrix can be decomposed into four possible extrinsic matrices ($[R|t]$) for the second camera. To find the correct one, we use the **cheirality**

condition. This physical constraint requires that the reconstructed 3D point must be in front of both cameras (i.e., have a positive depth value in both camera coordinate systems). Our implementation triangulates the points using each of the four candidate matrices and counts how many points have a positive depth for both views. The matrix that yields the highest count of valid points is selected as the correct one.

1.4.2 Re-projection Error

After triangulation, the re-projection error was computed by projecting the 3D points back onto the first image using M_1 and calculating the mean Euclidean distance between the re-projected points and the original ground-truth points. The implementation successfully achieved a re-projection error of less than 2 pixels, confirming the accuracy of the reconstruction pipeline.

1.5 Putting It All Together: Final Reconstruction

1.5.1 Results

The full pipeline was executed by the `test_temple_coords` script, which integrates all the previous functions. The script first calculates the correspondences for a denser set of points on the temple, then triangulates them to create a 3D point cloud.

The computed extrinsic parameters for the two cameras are:

```
--- Computed Extrinsic Parameters ---
R1 (Rotation of Camera 1):
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]

t1 (Translation of Camera 1):
[0. 0. 0.]

R2 (Rotation of Camera 2):
[[ 0.9656441 -0.0240544  0.25875252]
 [ 0.02357884 0.99970977 0.00494161]
 [-0.25879629 0.00132924 0.96593101]]

t2 (Translation of Camera 2):
[-1.          -0.02044277  0.07758157]
```

The final 3D reconstruction of the temple is visualized from three different angles in Figure 3.

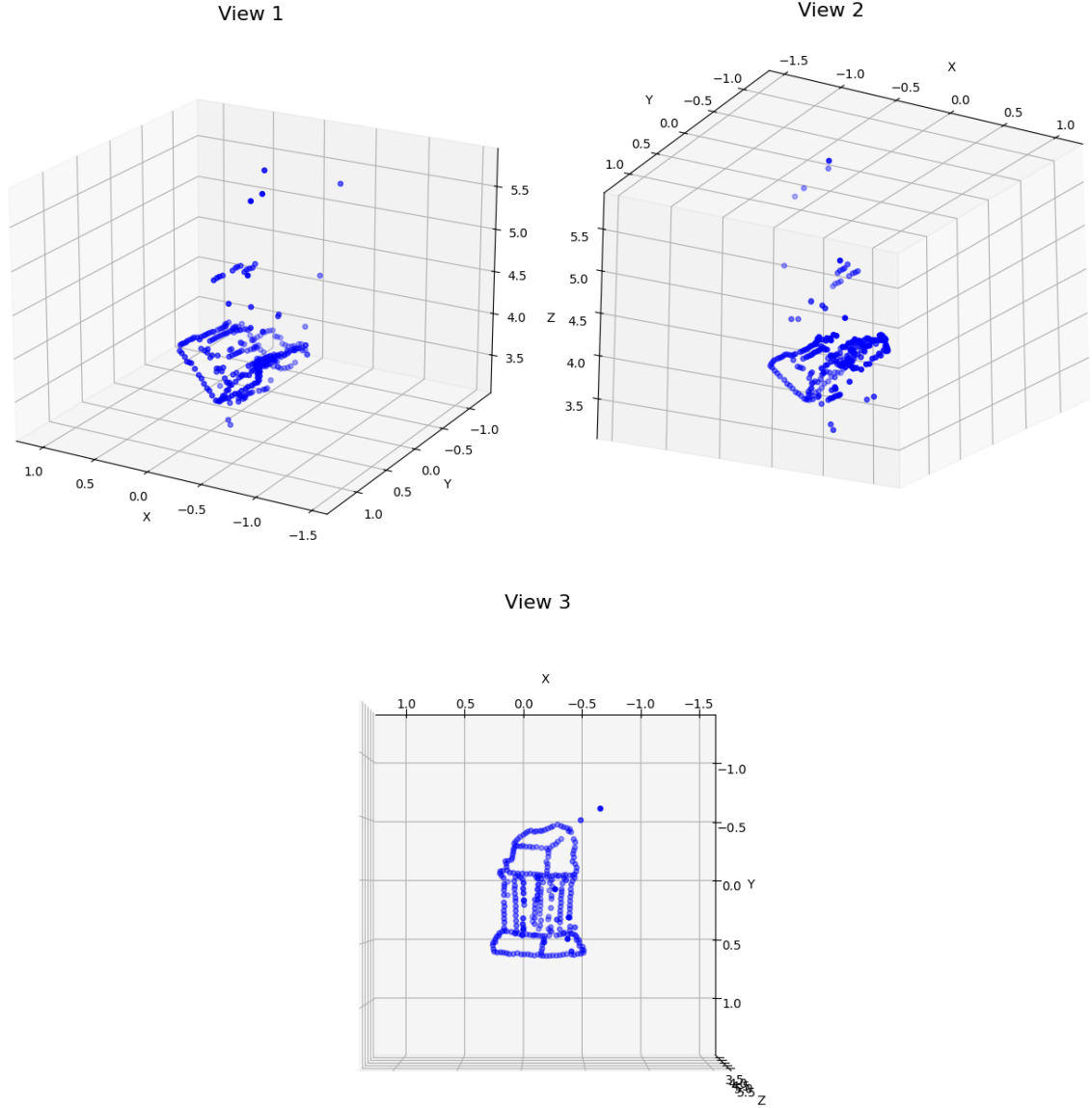


Figure 3: Final 3D reconstruction of the temple points from three different viewpoints.

2 Part 2: Pose Estimation

In this part, we implemented functions to estimate the full camera projection matrix (M) and to decompose it into its intrinsic (K) and extrinsic (R, t) parameters, given a set of 2D image points and their corresponding 3D world points.

2.1 Estimating M

2.1.1 Theory and Implementation

The function `estimate_pose` computes the 3×4 camera matrix M using the Direct Linear Transform (DLT) algorithm. Each correspondence between a 3D point P_i and a 2D point p_i provides two linear equations on the elements of M . With at least 6 correspondences, we can form a homogeneous system $Am = 0$, where m is the 12×1 vectorized camera matrix. We solve this system using SVD, where the solution is the singular vector corresponding to the smallest singular value.

2.1.2 Results

The output from the provided test script is as follows. The extremely low reprojection and pose errors for the clean data validate the correctness of the DLT implementation.

```
Reprojection Error with clean 2D points: 3.5636727699721745e-10
Pose Error with clean 2D points: 1.9368430103615735e-12
Reprojection Error with noisy 2D points: 8.76035177515788
Pose Error with noisy 2D points: 1.0711090367374574
```

2.2 Intrinsic/Extrinsic Parameters

2.2.1 Theory and Implementation

The function `estimate_params` decomposes the camera matrix M into K , R , and t .

1. **Finding Camera Center (c):** The camera center is computed as the null space of M using SVD.
2. **Finding K and R :** The left 3×3 submatrix of M is $M_{sub} = KR$. We use **RQ decomposition** to factor M_{sub} into an upper-triangular matrix K and an orthonormal (rotation) matrix R .
3. **Resolving Ambiguities:** The decomposition is not unique. We enforce two constraints: (1) The diagonal elements of K (focal lengths) must be positive. (2) The determinant of R must be $+1$ (a rotation, not a reflection). We apply a correction matrix to ensure these conditions hold.
4. **Finding Translation (t):** The translation is computed via the formula $t = -Rc$.

2.2.2 Results

The output from the test script below shows near-zero error for the clean data, confirming that the decomposition logic, including the ambiguity resolution, is correct.

```
Intrinsic Error with clean 2D points: 2.4297498586407777e-12
Rotation Error with clean 2D points: 9.421000222595044e-13
Translation Error with clean 2D points: 1.822575632078397e-12
Intrinsic Error with noisy 2D points: 0.7619502292319961
Rotation Error with noisy 2D points: 0.03134685208849778
Translation Error with noisy 2D points: 0.03758868324237813
```

2.3 Visualizing Projection of a CAD Model

2.3.1 Results

Using our implemented `estimate_pose` and `estimate_params` functions, we estimated the camera pose from a real image of an airplane and its corresponding 3D CAD model. The results are visualized in Figure 4.

We can see that the projection appears highly accurate.

- In the first subplot, the projected 3D keypoints (black dots) align almost perfectly with the given 2D keypoints (green circles). This indicates a very low reprojection error and a successful pose estimation.
- In the second subplot, the CAD model is shown in 3D, rotated according to our estimated rotation matrix R .
- In the third subplot, the complete CAD model is projected back onto the image using our estimated camera matrix M . The red wireframe of the plane aligns extremely well with the airplane in the photograph, confirming that both the intrinsic and extrinsic parameters were estimated correctly.

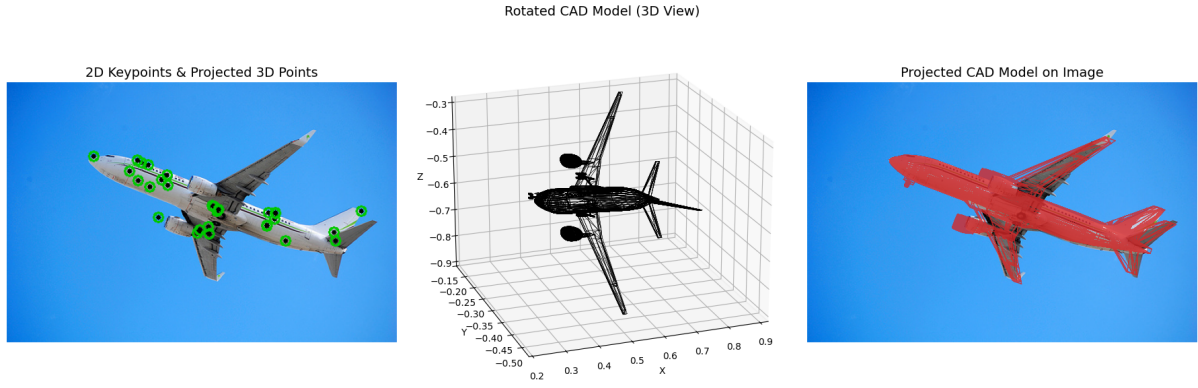


Figure 4: Visualization of camera pose estimation. (Left) Original 2D keypoints (green) and projected 3D keypoints (black). (Center) Rotated 3D CAD model. (Right) CAD model wireframe projected onto the original image.